Graph M4ker

Guía del código para el equipo
Cómo funciona cada clase y cada función

Esta guía recorre el proyecto Flutter `graph_maker_app_2` carpeta por carpeta. Para cada archivo explica qué hace la clase, qué guarda y qué hace cada función, con ejemplos de cómo se conectan entre sí. Está pensada para leerse de principio a fin la primera vez y luego usarse como referencia.

Rama: `hungarian-screen+johnson-screen` · Fecha: 17 de octubre de 2026
Dependencias principales: `provider`, `http`, `shared_preferences`, `flutter_colorpicker`, `tutorial_coach_mark`

Índice

1. Visión general y arquitectura
2. Punto de entrada: `main.dart`
3. Modelos (`models/`)
4. Reglas de edición (`rules/`)
5. Estado: `GraphProvider`
6. Servicios (`services/`)
7. Algoritmos (`algorithms/`)
8. Widgets del lienzo
9. Diálogos
10. Pantallas de grafos
11. Pantallas de ordenamiento
12. Tema y colores
13. Flujos completos paso a paso
14. Observaciones y puntos a revisar
15. Cómo agregar cosas nuevas

1. Visión general y arquitectura

Graph M4ker es una app Flutter para **dibujar grafos dirigidos y ponderados** y ejecutar algoritmos sobre ellos (asignación con el método húngaro y caminos máximos/mínimos tipo Johnson/CPM). También incluye **visualizaciones animadas de algoritmos de ordenamiento** y un **chat con IA** que conoce el grafo abierto.

El código sigue una separación por capas sencilla:

screens/
Pantallas completas (Scaffold). Deciden qué se muestra y reaccionan a los botones.

widgets/
Piezas reutilizables: el lienzo del grafo, el pintor, diálogos, tour de ayuda.

providers/
`GraphProvider`: guarda el grafo actual, valida con reglas, deshacer, guardar/abrir.

rules/
Qué se permite editar en cada modo (normal, asignación, Johnson).

services/
Preparan datos y llaman a los algoritmos; almacenamiento y API de IA.

algorithms/
Matemática pura: húngaro, Floyd-Warshall (Johnson1), Johnson clásico.

models/
Datos: `Graph`, `Node`, `Edge`, `MatrixData`, overlays…

theme/
Colores centralizados en `AppColors`.

assets/
`app_knowledge.md` (contexto para la IA) e ícono.

> **Idea clave:** la interfaz nunca modifica el `Graph` directamente. Siempre llama a un método de `GraphProvider`, que (1) consulta las `GraphRules` del modo actual, (2) guarda una copia para «deshacer», (3) aplica el cambio al `Graph` y (4) llama a `notifyListeners()` para que la pantalla se redibuje.

Grafo principal vs. copias de trabajo

Al iniciar la app se crea **un** `GraphProvider` global (el grafo principal). Cuando se abre la pantalla de Asignación o de Johnson, se crea **otro** `GraphProvider` con una *copia* (`snapshot()`) del grafo principal y con reglas más estrictas (`AssignmentRules` o `JohnsonRules`). Lo que se edite ahí no afecta al grafo principal hasta que el usuario pulsa «Apply to main graph» (o elige «Apply & Exit» al salir).

2. Punto de entrada: `main.dart`

`lib/main.dart`

Elemento	Qué hace
`main()`	Arranca la app envolviéndola en un `ChangeNotifierProvider` que crea el `GraphProvider` principal. Todo lo que esté debajo puede leerlo con `context.read/watch<GraphProvider>()`.
`MyApp`	`StatelessWidget` que construye el `MaterialApp`: título, tema claro y oscuro (Material 3, color semilla índigo) y la pantalla inicial `HomeScreen`. Oculta el banner de debug.

3. Modelos (`models/`)

Los modelos son clases de datos. Casi no tienen lógica, salvo `Graph`, que concentra las operaciones geométricas y de construcción de matrices.

3.1 `Node`

`lib/models/node.dart`

Representa un vértice dibujado en pantalla.

Campo / método	Descripción
`id`	Identificador interno único, formato `"n0"`, `"n1"`… Nunca cambia.
`name`	Nombre visible (A, B, …, Z, AA…). El usuario puede cambiarlo.
`x`, `y`	Posición del centro en el lienzo (píxeles lógicos).
`radius`	Radio del círculo (por defecto 30, viene de `Graph.radius`).
`color`	Color como entero ARGB (`0xFF2196F3` = azul por defecto, constante `defaultColor`). Se guarda como `int` para poder serializarlo a JSON fácilmente.
`rename(newName)`	Cambia `name`.
`reColor(newColor)`	Cambia `color`.

3.2 `Edge`

`lib/models/edge.dart`

Representa una arista entre dos nodos.

Campo / método	Descripción
`id`	Identificador único, formato `"e0"`, `"e1"`…
`sourceNodeId`	Id del nodo de origen.
`targetNodeId`	Id del nodo de destino.
`weight`	Peso / costo (`double`).
`directed`	`true` = flecha de origen a destino; `false` = arista no dirigida (vale en ambos sentidos).
`isLoop()`	**Nuevo.** Devuelve `true` si la arista es un *bucle* (self-loop), es decir, si `sourceNodeId == targetNodeId`. Centraliza esa comparación que antes se repetía en varios archivos.

> **Sobre `isLoop`:** en el código actual está declarado como **método** (`bool isLoop() => …`), por eso se llama con paréntesis: `edge.isLoop()`. Si se quiere usar como *getter* (`edge.isLoop` sin paréntesis) habría que declararlo `bool get isLoop => sourceNodeId == targetNodeId;` y quitar los paréntesis en `graph.dart` y `graph_painter.dart`. Se usa en:
>
> - `Graph.findEdgeAt` – para detectar toques sobre el círculo del bucle.
> - `Graph.buildMatrixData` – para no contar dos veces un bucle no dirigido.
> - `GraphPainter.paint` – para dibujar el bucle como círculo en vez de línea.
>
> Todavía hay sitios que comparan a mano (`AssignmentRules.validateGraph`, `JohnsonRules.validateGraph`, y `fromId == toId` en las reglas); se podrían migrar a `isLoop()`.

3.3 `Graph`

`lib/models/graph.dart`

Contenedor de nodos y aristas, con toda la lógica de geometría, serialización y matriz de adyacencia.

Campos

Campo	Descripción
`nodes`	`Map<String, Node>` indexado por id.
`edges`	`Map<String, Edge>` indexado por id.
`currentNode`, `currentEdge`	Contadores para generar ids nuevos. Solo aumentan (si borras `n2`, el siguiente nodo seguirá siendo `n3`), así los ids nunca se repiten.
`radius`	Radio que se asigna a los nodos nuevos (30).
`selfLoopRadius` (20)	Radio del círculo que dibuja un bucle.
`overLapDistance` (10)	Cuánto se solapa el círculo del bucle con el nodo.
`_parallelEdgeOffset` (7.5)	Separación lateral cuando existen A→B y B→A, para que no se dibujen encima.

Métodos de edición

Método	Qué hace
`addNode(x, y)`	Crea un nodo con id `"n{currentNode}"`, nombre de `generateNodeName()` y lo guarda. Incrementa el contador.
`removeNode(id)`	Primero borra todas las aristas que entran o salen del nodo (para no dejar aristas «huérfanas»), luego borra el nodo.
`addEdge(src, tgt, weight, {directed})`	Crea una arista con id `"e{currentEdge}"`.
`removeEdge(id)`	Elimina la arista del mapa.
`moveNode(id, x, y)`	Actualiza la posición del nodo (si existe).
`renameNode(id, name)` / `recolorNode(id, color)`	Delegan en `Node.rename` / `Node.reColor`.
`updateEdgeWeight(id, w)`	Cambia el peso de una arista.
`clear()`	Vacía nodos y aristas y reinicia los contadores a 0.

Métodos de búsqueda / geometría

Método	Qué hace
`findNode(id)`	Devuelve el nodo o `null`.
`findNodeAt(x, y)`	Devuelve el id del nodo cuyo círculo contiene el punto (distancia^2 ≤ radio^2), o `""` si no hay ninguno. Compara distancias al cuadrado para evitar la raíz cuadrada.
`findEdges(id)`	Lista de aristas que **salen** del nodo `id`. Lo usa `JohnsonRules` para recorrer el grafo.
`findEdgeAt(x, y)`	Devuelve el id de la arista tocada (tolerancia 10 px) o `""`. Para un bucle mide la distancia al borde del círculo del bucle. Para una arista normal calcula el segmento (desplazado lateralmente si existe la arista inversa, igual que en el dibujo) y usa `_distanceToSegment`.
`_distanceToSegment(px,py,x1,y1,x2,y2)`	Proyecta el punto sobre el segmento (parámetro `t` limitado a [0,1]) y devuelve la **distancia al cuadrado** al punto más cercano.
`selfLoopCenterDistance(radius)` (static)	Distancia del centro del nodo al centro del círculo del bucle: `radius + 20 − 10`. La comparten el pintor y la detección de toques para que coincidan.
`findReverseEdge(src, tgt)`	Devuelve la arista que va de `tgt` a `src`, si existe.
`edgeExists(src, tgt)`	`true` si ya hay una arista exactamente de `src` a `tgt`.

Copias y serialización

Método	Qué hace
`clone()`	Copia **profunda**: crea nodos y aristas nuevos. Es la base de «deshacer» y de las copias de trabajo. Si se agrega un campo a `Node` o `Edge`, **hay que añadirlo aquí**.
`toJson()`	Convierte el grafo completo a un `Map` (contadores, radio, listas de nodos y aristas). Se usa para guardar y para comparar si hubo cambios.
`fromJson(json)` (static)	Reconstruye un `Graph` a partir del `Map`. Tolera campos faltantes (usa valores por defecto con `??`).
`toSummaryMap()`	Resumen reducido (nombres de nodos y aristas con from/to/peso) que se envía como contexto al chat de IA.

Matriz y nombres

Método	Qué hace
`buildMatrixData()`	Construye la matriz de adyacencia **rectangular**: las filas son solo los nodos que tienen aristas salientes y las columnas solo los que tienen aristas entrantes (una arista no dirigida cuenta en ambos sentidos). Para cada celda guarda el peso, si hay arista (`hasEdge`) y su id (`edgeIds`). Además calcula suma por fila/columna y grado (número de aristas) por fila/columna. Un bucle no dirigido se cuenta una sola vez gracias a `isLoop()`. Devuelve un `MatrixData`.
`adjacencyMatrix(matrixData)`	Atajo que devuelve `matrixData.matrix`.

`generateNodeName()`	Convierte `currentNode + 1` a letras estilo columnas de Excel: 1→A, 26→Z, 27→AA, 28→AB… (base 26 «biyectiva» usando una pila).

3.4 `MatrixData`

`lib/models/matrix_data.dart`

Objeto inmutable con el resultado de `buildMatrixData()`.

Campo	Descripción
`nodes`	Todos los nodos del grafo.
`rowHeaders` / `colHeaders`	Nodos que forman filas (orígenes) y columnas (destinos).
`matrix`	`matrix[i][j]` = peso de la arista fila i → columna j (0 si no hay).
`hasEdge`	Distingue «peso 0» de «no hay arista».
`edgeIds`	Id de la arista en cada celda (`""` si no hay). Sirve para resaltar aristas desde un resultado.
`rowSum`, `colSum`	Suma de pesos por fila / columna («Attributes leaving/arriving»).
`rowDegree`, `colDegree`	Cantidad de aristas por fila / columna («Grade»).

3.5 `AppViewport`

`lib/models/app_viewport.dart`

Preparado para zoom y desplazamiento del lienzo. Guarda `offset` y `scale` (entre 0.25 y 3.0). **Actualmente se crea en `GraphWidget` pero el pintor no lo aplica** (el código de la cuadrícula/transformación está comentado), así que hoy no tiene efecto visible.

Método	Qué hace
`toWorld(p)`	Pantalla → mundo: `(p - offset) / scale`.
`toScreen(p)`	Mundo → pantalla: `p * scale + offset`.
`panBy(delta)`	Desplaza la vista.
`zoomAt(focal, newScale)`	Cambia el zoom manteniendo fijo el punto bajo el dedo.
`zoomBy(focal, factor)`	Zoom relativo (multiplica la escala).
`reset()`	Vuelve a offset 0 y escala 1.
`fitToRect(rect, size, padding)`	Calcula escala y offset para que un rectángulo del mundo quepa centrado en la vista (máx. escala 1.5).

3.6 `AssignmentOverlay` y clases auxiliares

`lib/models/assignment_overlay.dart (archivo nuevo)`

Datos de «solo visualización» que la pantalla de Asignación dibuja **encima** del grafo sin modificarlo (nodos ficticios, aristas fantasma, pares asignados).

Clase	Descripción

`AssignmentPairKind`	Enum: `real` (arista real asignada), `dummy` (asignada a un nodo ficticio), `noEdge` (el algoritmo emparejó dos nodos sin arista entre ellos).
`AssignmentPair`	Un par del resultado: nombre de origen, nombre de destino, peso y tipo. Se lista en la hoja de resultados.
`GhostNode`	Nodo ficticio (D1, D2…) con posición y radio, dibujado gris y punteado.
`GhostEdge`	Arista punteada entre nodos (reales o fantasma). `assigned` = forma parte de la solución; `noEdge` = asignación sin arista real (se dibuja en rojo).
`AssignmentOverlay`	Agrupa las listas anteriores y `highlightedEdgeIds` (aristas reales a resaltar en verde).

3.7 Otros modelos

Clase	Descripción
`ChatMessage` / `ChatRole`	`chat_message.dart` Un mensaje del chat con su rol (`user` o `assistant`) y texto.
`BoloPainter`	`bolo.dart` `CustomPainter` que dibuja un bolo (pin de bolos) con curvas Bézier sobre un lienzo base de 100×300, lo escala al tamaño real, dibuja el contorno, la franja roja y el número en el centro. `shouldRepaint` solo repinta si cambian color o número.
`Bolo`	Widget que envuelve `BoloPainter`. `escala` controla el tamaño (1.0 = 100×300). Lo usan las pantallas de ordenamiento: **un bolo más alto = valor más grande**.

4. Reglas de edición (`rules/`)

Las reglas aplican el patrón **Strategy**: `GraphProvider` recibe un objeto `GraphRules` y le pregunta antes de cada cambio. Cada método `check…` devuelve `null` si la acción está permitida, o un `String` con el mensaje de error que se muestra en un SnackBar.

4.1 `GraphRules` (abstracta) y `DefaultRules`

`lib/rules/graph_rules.dart`

También define el enum `Mode { normal, assignment, johnson }` (hoy no se usa en otro sitio).

Método	Comportamiento por defecto
`forcesDirected()`	`false`. Si es `true`, el diálogo de arista oculta la opción «Non-directed».
`checkAddNode(graph, x, y)`	Rechaza el nodo si queda a menos de `2·radio + 3` de otro (evita solapes).
`checkMoveNode(graph, id, x, y)`	Igual que el anterior, pero ignorando al propio nodo. Error si el nodo no existe.
`checkEdgeEndpoints(graph, from, to)`	Rechaza si la arista ya existe, o si existe la inversa y es no dirigida (porque ya cubre ambos sentidos). Se llama al **soltar el arrastre**, antes de pedir el peso.
`checkAddEdge(graph, from, to, directed)`	Llama a `checkEdgeEndpoints`. Se llama al **guardar** el peso.
`checkRemoveNode` / `checkRemoveEdge`	Siempre permiten (`null`).
`validateGraph(graph)`	Lista de problemas del grafo completo; vacía por defecto. Se usa al abrir un grafo guardado y en las pantallas de algoritmos.

`DefaultRules` es simplemente `GraphRules` sin cambios (modo editor libre).

4.2 `AssignmentRules`

`lib/rules/assignment_rules.dart`

Garantiza un grafo **bipartito dirigido**: cada nodo es solo origen o solo destino.

Método	Regla adicional
`forcesDirected()`	`true`.
`checkEdgeEndpoints`	Primero las reglas base. Luego: no se permiten bucles; el origen no puede ser ya destino de otra arista; el destino no puede ser ya origen de otra arista.
`checkAddEdge`	Además rechaza aristas no dirigidas.
`validateGraph`	Recorre todas las aristas y devuelve el primer problema encontrado: arista no dirigida, bucle, o un nodo con aristas entrantes y salientes.

4.3 `JohnsonRules`

`lib/rules/johnson_rules.dart`

Garantiza un **grafo dirigido acíclico (DAG)**, necesario para calcular caminos máximos.

Método	Regla adicional
`forcesDirected()`	`true`.
`checkEdgeEndpoints`	Sin bucles, sin aristas inversas, y **sin ciclos**: hace un recorrido en anchura (BFS con `Queue`) desde `to`; si llega a `from`, la nueva arista cerraría un ciclo y se rechaza.
`checkAddEdge`	Rechaza aristas no dirigidas.
`validateGraph`	Primer problema: arista no dirigida o bucle. (Los ciclos se detectan después, en `JohnsonScreen`.)

5. Estado: `GraphProvider`

`lib/providers/graph_provider.dart`

Es un `ChangeNotifier`: cuando cambia, todos los widgets que hicieron `context.watch<GraphProvider>()` se reconstruyen. Es la **única puerta** para modificar el grafo.

Campos

Campo	Descripción
`_graph`	El grafo actual.
`rules`	Reglas del modo (`DefaultRules` si no se indica).
`_history`	Pila de copias para deshacer; máximo `_maxHistory = 20` (se descarta la más antigua).
`_storage`	`GraphStorageService` para guardar/abrir.

Constructor: `GraphProvider({Graph? initialGraph, GraphRules? graphRules})`. Las pantallas de algoritmos lo crean con la copia del grafo principal y sus reglas.

Patrón de los métodos que modifican

```
String? addNode(double x, double y) {
  String? message = rules.checkAddNode(_graph, x, y);   // 1. validar
  if (message != null) return message;                  //    → error a la UI
  _pushHistory();                                       // 2. guardar para deshacer
  _graph.addNode(x, y);                                 // 3. aplicar
  notifyListeners();                                    // 4. redibujar
  return null;                                          //    → éxito
}
```

Método	Qué hace
`graphGetter()`	Devuelve el `Graph` actual (para leer, no para modificar).
`snapshot()`	Devuelve un `clone()` del grafo.
`replaceGraph(g)`	Sustituye el grafo por una copia de `g` (con historial). Se usa al aplicar cambios desde una copia de trabajo.
`addNode`, `removeNode`, `moveNode`	Patrón validar → historial → aplicar → notificar. Devuelven mensaje o `null`.
`addEdge(src, tgt, w, directed)`	Igual, y además: si se crea una arista **no dirigida** y ya existía la inversa, borra la inversa (la no dirigida la reemplaza).
`removeEdge(id)`	Valida y elimina.
`renameNode`, `recolorNode`, `updateNode`, `updateEdgeWeight`	Cambios sin reglas (siempre permitidos), con historial y notificación. `updateNode` cambia nombre y color a la vez (usado por el diálogo de editar nodo).

`clearGraph()`	Vacía el grafo (se puede deshacer).
`canUndo` (getter)	`true` si hay historial. Habilita el botón «Undo».
`_pushHistory()`	Apila un `clone()` del grafo actual.
`undo()`	Restaura la última copia del historial.
`findNodeAt`, `findEdgeAt`, `findReverseEdge`, `edgeExists`	Delegan en `Graph`.
`checkEdgeEndpoints(src, tgt)`	Delegan en `rules` (se llama al soltar un arrastre).
`validateGraph()`	Delegan en `rules.validateGraph`.
`buildMatrixData()`, `adjacencyMatrix()`	Delegan en `Graph`.
`graphSummaryJson()`	JSON en texto de `toSummaryMap()`, para la IA.
`listSavedGraphs()`, `savedGraphExists(name)`	Consultas al almacenamiento.
`saveGraph(name)`	Guarda `toJson()` con ese nombre.
`loadGraph(name)`	Carga, reconstruye con `fromJson` y **valida con las reglas del modo**: así no se puede abrir, por ejemplo, un grafo con ciclos dentro de Johnson. Devuelve mensaje de error o `null`.
`deleteGraph(name)`	Borra un grafo guardado.

6. Servicios (`services/`)

6.1 `GraphStorageService`

`lib/services/graph_storage_service.dart`

Guarda todos los grafos en **una sola entrada** de `SharedPreferences` (clave `"saved_graphs"`), como un JSON `{ "nombre": grafoJson, … }`.

Método	Qué hace
`_readAll()`	Lee y decodifica el mapa completo (vacío si no hay nada).
`_writeAll(all)`	Codifica y escribe el mapa completo.
`listNames()`	Nombres guardados, ordenados alfabéticamente.
`exists(name)`	¿Existe un grafo con ese nombre?
`save(name, json)`	Lee todo, agrega/reemplaza la entrada y escribe.
`load(name)`	Devuelve el JSON de ese grafo o `null`.
`delete(name)`	Quita la entrada y escribe.

6.2 `AiService`

`lib/services/ai_service.dart`

Cliente HTTP para el asistente de IA vía la API de Groq (compatible con OpenAI), modelo `openai/gpt-oss-20b`.

Elemento	Descripción
`AiServiceException`	Excepción con un mensaje amigable para mostrar en el chat.
`_apiKey`	Se lee en tiempo de compilación: `flutter run --dart-define=GROQ_API_KEY=tu-clave`. Si está vacía, se lanza un error explicándolo.
`_loadDocumentation()`	Carga `assets/docs/app_knowledge.md` una sola vez y la guarda en caché (`_cachedDocs`).
`sendMessage({userMessage, history, graphSummaryJson})`	Arma el *system prompt* (rol del asistente + documentación + estado del grafo en JSON), agrega el historial y el mensaje nuevo, hace el `POST` con timeout de 30 s y devuelve el texto de la respuesta. Traduce errores (timeout, sin internet, error HTTP, respuesta vacía) a `AiServiceException`.

6.3 `AssignmentService`

`lib/services/assignment_service.dart`

Puente entre el grafo (como `MatrixData`) y el algoritmo húngaro. `mode` = `true` para **minimizar**, `false` para **maximizar**.

Método	Qué hace
`prepare()`	Crea la matriz de costos. Donde no hay arista pone una **penalización** `M` = 1 + suma de |pesos|: `+M` al minimizar y `−M` al maximizar, para que el algoritmo evite esas celdas siempre que pueda.
`run()`	Ejecuta `HungarianAssignment().solve(prepare(), mode)` y recalcula `totalCost` sumando **solo** pesos de aristas reales (ignora penalizaciones y ficticios).
`buildOverlay(result)`	Construye el `AssignmentOverlay`. Si la matriz no era cuadrada, crea nodos fantasma (D1, D2…) en el lado que falta, dibuja aristas fantasma desde/hacia ellos (marcando la asignada), y para cada par del resultado decide si es `real` (resalta la arista), `dummy`, o `noEdge` (agrega una arista fantasma roja).
`padToSquare(data)` (static)	Devuelve una copia de `MatrixData` rellenada con filas/columnas ficticias (id `dummy_k`) hasta ser cuadrada. Lo usa `MatrixScreen(square: true)`.
`isDummy(node)` (static)	`true` si el id empieza con `"dummy_"`.
`_placeGhosts(side, count)`	Coloca los nodos fantasma debajo del nodo más bajo del lado correspondiente, bajando de a un paso (`2·radio + 15`) hasta encontrar un lugar libre que no choque con nodos reales ni con otros fantasmas.

6.4 `JohnsonService`

`lib/services/johnson_service.dart`

Prepara los datos, ejecuta `Johnson1` (Floyd-Warshall) y calcula los valores tipo **CPM / ruta crítica**: tiempos hacia atrás y holguras. También define `enum Objective { max, min }` y la clase `JohnsonData` (matriz `cost` + matriz `edgesIds`).

Método / campo	Qué hace
`nodeFrom`, `nodeTo`	Nodo inicio (solo tiene salidas) y nodo fin (solo tiene entradas).
`index`	Mapa id de nodo → índice en las matrices.
`prepare()`	Elimina nodos aislados (sin aristas), recalcula inicio y fin, crea la matriz de costos (0 en la diagonal, ∞ donde no hay arista, el peso donde sí) y la matriz de ids de aristas.
`run()`	1. Llama a `Johnson1.solve`. 2. `distanceMax` viene calculada con costos negados, así que la vuelve a negar para obtener los **caminos máximos** reales. 3. `t` = duración total = camino máximo de inicio a fin. 4. `backwards[nodo]` = `t − (máximo de nodo a fin)` → «tiempo más tardío» de cada nodo. 5. `holgura[arista]` = `backwards[destino] − (máximo de inicio a origen) − peso`. Holgura 0 = arista en la ruta crítica. Se muestra como `h = …` junto al peso.
`path(obj, {from, to})`	Reconstruye el mejor camino (máx. o mín.) caminando **hacia atrás** con `bestEdgeIds[i][j]` (la última arista del camino i→j). Devuelve un `Set` con los ids de aristas y, **como último elemento, la distancia total en texto** (convención que usa `_buildTextPath` en la pantalla). Devuelve vacío si no hay camino.

7. Algoritmos (`algorithms/`)

7.1 `HungarianAssignment` y `HungarianResult`

`lib/algorithms/hungarian_assignment.dart`

Método	Qué hace
`solve(matrix, min)`	Calcula `n = max(filas, columnas)`, rellena a cuadrada (`addDummies`), y si se maximiza transforma la matriz (`_negate`). Ejecuta `_kuhnMunkres`. Llena el resultado: `fullAssignment` (incluye ficticios), `assignment` (por fila real, −1 si quedó con columna ficticia), `rows`, `cols`, `totalCost`.
`addDummies(matrix, rows, cols)`	Matriz n×n: valores originales y 0 en las celdas nuevas.
`_negate(matrix)`	Convierte maximizar en minimizar: `max - valor` en cada celda (así no hay números negativos).
`_kuhnMunkres(a)`	Implementación clásica O(n^3) del método húngaro con **potenciales**: u (filas), v (columnas), `p[j]` = fila asignada a la columna j, `way` = camino para reasignar. Agrega las filas una por una; para cada una busca la columna libre de menor costo reducido (`a - u - v`), ajusta potenciales con `delta` y al encontrar una columna libre «voltea» el camino aumentante. Devuelve `assignment[fila] = columna`.

`HungarianResult`: `assignment`, `totalCost`, `fullAssignment`, `rows`, `cols`.

7.2 `Johnson1` y `JohnsonResult1` (el que usa la app)

`lib/algorithms/johnson1.dart`

A pesar del nombre, calcula todas las distancias con **Floyd-Warshall** (válido porque el grafo es un DAG, gracias a `JohnsonRules`).

Método	Qué hace
`solve(cost, edgeIds, nodes)`	Ejecuta Floyd-Warshall dos veces: con costos negados (→ caminos máximos, aún negativos) y con los costos normales (→ mínimos). Junta ambos en un solo `JohnsonResult1`.
`floydWarshall(cost, nodes, edgeIds)`	Triple bucle clásico: para cada nodo intermedio k, si `d[i][k] + d[k][j] < d[i][j]` actualiza la distancia y copia `bestEdgeIds[i][j] = bestEdgeIds[k][j]` (la última arista del camino). Devuelve el resultado en los campos `...Max`.
`negate(matrix)`	Cambia de signo todo menos ∞.

`JohnsonResult1`: `distanceMax`, `bestEdgeIdsMax`, `distanceMin`, `bestEdgeIdsMIn`, `backwards`, `holgura`.

7.3 `JohnsonAlgorithm` y `JohnsonResult` (no usados actualmente)

`lib/algorithms/johnson.dart · lib/algorithms/johnson_result.dart`

Implementación del **algoritmo de Johnson real**, con nodos por índice. Ninguna pantalla lo importa hoy; queda como alternativa más general (detecta ciclos y funciona con grafos no acíclicos).

Elemento	Descripción
`JohnsonEdge`	Arista con `from`, `to` (índices) y `weight`.
`JohnsonAlgorithm.run({nodeCount, edges, objective})`	1) Bellman-Ford desde un nodo virtual para obtener potenciales `h` y detectar ciclos negativos; 2) repondera `w' = w + h(u) − h(v)` (≥ 0); 3) Dijkstra $O(n^2)$ desde cada nodo; 4) deshace la reponderación. Para maximizar, niega los pesos al inicio y al final.
`PathObjective`	Enum `minimize` / `maximize`.
`JohnsonResult`	`dist`, `pred` (predecesores), `potentials`, `hasNegativeCycle`. Métodos: `JohnsonResult.cycle(...)` (resultado vacío por ciclo), `cycleMessage`, `isReachable(from,to)`, `pathBetween(from,to)` (lista de nodos del camino).

8. Widgets del lienzo

8.1 `GraphWidget` (lienzo interactivo)

`lib/widgets/graph_canvas.dart`

Widget con estado que traduce gestos en llamadas al `GraphProvider` y dibuja todo con `GraphPainter`. Parámetros opcionales para las pantallas de algoritmos: `highlightedEdgeIds`, `highlightedColor`, `holgura`, `ghostNodes`, `ghostEdges`.

Gestos

Gesto	Resultado
Toque en vacío	`addNode` (muestra error si está muy cerca de otro).
Toque en un nodo	Abre `showEditNodeDialog` (nombre y color).
Arrastrar desde un nodo y soltar en otro	Dibuja una línea de vista previa; al soltar valida con `checkEdgeEndpoints` y abre `showEdgeDialog`. Soltar en el mismo nodo crea un bucle (si las reglas lo permiten).
Mantener pulsado un nodo 500 ms y arrastrar	Modo **mover nodo** (vibración leve). Al soltar llama a `moveNode`.
Doble toque en nodo / arista	Elimina el nodo (con sus aristas) o la arista.

Estado interno y callbacks

Elemento	Descripción
`sourceNode`, `currentDragPosition`	Nodo desde el que se arrastra y posición actual del dedo (vista previa de arista).
`_holdTimer`, `_panMoved`	Temporizador de pulsación larga; si el dedo se movió antes de 500 ms, no entra en modo mover.
`_isMovingNode`, `_movingNodeId`, `_movingNodeCurrentPosition`	Estado del modo mover; el pintor dibuja el nodo en la posición temporal.
`_suppressNextTap`	Evita que, al terminar una pulsación larga, se dispare también el toque (que abriría el diálogo).
`_cancelHold()`	Cancela el temporizador.
`onPanDown`	Detecta si se tocó un nodo y arranca el temporizador de pulsación larga.
`onPanStart` / `onPanUpdate`	Marca movimiento y actualiza la posición (de la vista previa o del nodo que se mueve).
`onPanEnd`	Termina el movimiento o la creación de arista.
`onPanCancel`	Resetea todo el estado.
`onDoubleTapDown`	Borrar nodo o arista.
`onTapUp`	Crear nodo o editar nodo.

`dispose()`	Cancela el temporizador al destruir el widget.

8.2 `GraphPainter`

`lib/widgets/graph_painter.dart`

`CustomPainter` que dibuja en este orden: **aristas** → **aristas fantasma** → **línea de vista previa** → **nodos** → **nodos fantasma** (lo último queda encima).

Método	Qué hace
`_positionOf(node)`	Posición del nodo, o la temporal si se está moviendo.
`_drawArrowhead(canvas, tip, ux, uy, color)`	Triángulo de la flecha usando el vector unitario (`ux, uy`) y su perpendicular.
`_drawDirectedEdge(...)`	Línea desde el origen hasta el **borde** del nodo destino (no hasta el centro) y flecha si es dirigida.
`_drawSelfLoop(...)`	Dibuja el bucle como un círculo a la derecha del nodo (usa `Graph.selfLoopCenterDistance`).
`_drawDashedLine(...)`	Línea punteada (8 px trazo, 6 px espacio).
`_drawGhostEdges(canvas)`	Aristas del overlay: primero las no asignadas y luego las asignadas (para que queden arriba). Gris, color de resaltado, o rojo con texto «no edge».
`_drawGhostNodes(canvas)`	Círculos grises semitransparentes con borde punteado (16 arcos) y el nombre.
`paint(canvas, size)`	Para cada arista: si `isLoop()` → bucle + peso debajo. Si no → color según dirección (naranja izquierda→derecha, turquesa derecha→izquierda) o color de resaltado, grosor 5 u 8, desplazamiento si hay arista inversa, etiqueta con peso y holgura (`h = …`). Después la vista previa y los nodos; el color del texto del nodo se elige según la luminancia del fondo.
`shouldRepaint`	Siempre `true` (redibuja en cada build).

8.3 Ayuda y tour

Clase	Descripción
`GraphTour` / `TourStep`	`graph_tour.dart` Tour guiado con `tutorial_coach_mark`. Cada `TourStep` apunta a un widget por su `GlobalKey`, con título, texto, alineación y forma del foco (círculo o rectángulo redondeado). `GraphTour.show(context, pasos)` construye los `TargetFocus` y muestra el tutorial; `_Bubble` es la burbuja morada con «1/6», botones anterior/siguiente y ✓ al final.
`GraphHelp`	`graph_help.dart` Ventana de ayuda estática con la lista de botones (`_HelpItem`: ícono, título, descripción). En pantallas anchas (≥ 600 px) se muestra como `Dialog`; en móviles como hoja inferior. Actualmente no se llama desde ninguna pantalla (se reemplazó por el tour).
`MenuOption`	`menu_option.dart` Enum `save, open, clear, chat` para el menú «⋮» de las pantallas de algoritmos.

9. Diálogos (`widgets/dialogs/`)

Función	Qué hace
`showEdgeDialog(context, provider, source:, target:)`	Pide el peso (campo numérico) y, si las reglas lo permiten, si es dirigida o no (`ToggleButtons`, oculto cuando `forcesDirected()`). Usa `StatefulBuilder` para que el toggle se actualice dentro del diálogo. Al guardar llama a `provider.addEdge` y muestra el error si lo hay. Si el texto no es un número, no hace nada.
`showEditNodeDialog(context, provider, nodeId)`	Diálogo con dos pestañas: «Name» (texto) y «Color» (`ColorPicker`). Al guardar llama a `provider.updateNode`.
`showSaveDialog(context, provider)`	Pide un nombre; si ya existe pregunta «Replace?»; guarda y muestra «Saved as …». Comprueba `context.mounted` después de cada `await`.
`showOpenDialog(context, provider)`	Lista los grafos guardados. Tocar uno llama a `loadGraph` (si las reglas lo rechazan, muestra el error en rojo dentro del diálogo). El ícono de papelera lo borra.
`showConfirmExitCopyDialog(context, mainProvider, copy)`	Al salir de una pantalla con copia de trabajo modificada: **Discard** (salir sin aplicar), **Cancel** (quedarse), **Apply & Exit** (`mainProvider.replaceGraph(copy.snapshot())` y salir). Enum `ExitOption`.

10. Pantallas de grafos (`screens/`)

10.1 `HomeScreen` (menú principal)

`lib/screens/home.dart`

Pantalla inicial con el título «Graph M4ker», un acordeón de conceptos teóricos y un carrusel horizontal de tarjetas de métodos.

Elemento	Descripción
`_Concept` / `_concepts`	Lista de conceptos (título + texto) del acordeón. Para agregar uno, añade un `_Concept`.
`_Method` / `_methods`	Lista de tarjetas: nombre, descripción, ícono y función open que navega. Para agregar una pantalla al menú, añade un `_Method`.
`_push(context, page)`	Navega a una pantalla normal (usa el grafo principal).
`_pushWithCopy(context, rules, screen)`	Navega creando un **nuevo** `GraphProvider` con `main.snapshot()` y las reglas indicadas; pasa el provider principal a la pantalla para poder aplicar cambios.
`_HomeScreenState`	Guarda qué concepto (`_openConcept`) y qué tarjeta (`_openCard`) están abiertos; solo uno a la vez. Calcula tamaños responsivos (título, márgenes laterales).
`_ConceptTile`	Fila del acordeón: animación del «+» que gira a «×», sombra y `AnimatedSize` para mostrar el texto.
`_MethodCard`	Tarjeta con ícono grande; al tocarla se eleva y muestra la descripción y el botón «Abrir».

10.2 `GraphScreen` (editor libre)

`lib/screens/graph_screen.dart`

Editor principal sobre el grafo global con `DefaultRules`.

Elemento	Descripción
AppBar	Undo, Clear, Save, Open, Chat IA, Help (tour).
Botones flotantes	**J** → `JohnsonScreen` con copia + `JohnsonRules`; **HU** → `AssignmentScreen` con copia + `AssignmentRules`; **M[x]** → `MatrixScreen`; ⋯ → menú de ordenamientos.
`_startTour()`	Define los pasos del tour con las `GlobalKey` de cada botón.
`_showSortMenu(fabContext)`	Calcula la posición del botón con `RenderBox` para abrir el menú justo a su izquierda, y navega al ordenamiento elegido.
`createProvider(rules)`	Función local que crea el provider de la copia de trabajo.

10.3 `AssignmentScreen` (método húngaro)

`lib/screens/assignment_screen.dart`

Trabaja sobre una copia (`_copy`) con `AssignmentRules`.

Elemento	Descripción
`_lastResult`, `_lastOverlay`	Último resultado y overlay mostrados.
`_lastAppliedJson`	JSON del grafo la última vez que se aplicó al principal. Si el actual es distinto → `hasChanges` (habilita «Apply» y activa la confirmación al salir con `PopScope`).
`initState()`	Toma la copia del contexto y se suscribe con `_listenerClear`.
`_listenerClear()`	Si el grafo cambia, borra el resultado anterior (ya no sería válido).
`_runAssignment(min)`	`buildMatrixData` → `AssignmentService.run` → `buildOverlay`; guarda el resultado y abre una hoja inferior con la lista de pares (`A -> B peso`, «(dummy)», «(no edge)») y el total.
`build()`	Calcula `messages` (validación), `canSolve` (hay aristas y no hay errores), `hasChanges`. Muestra el lienzo con el overlay, el botón M[x] (matriz cuadrada con ficticios), el error si hay, y los botones Minimize / Maximize.
`handleMenuSelection`	Save / Open / Clear / ChatAI sobre la copia.

10.4 `JohnsonScreen` (caminos máximo/mínimo y holguras)

`lib/screens/johnson_screen.dart`

Trabaja sobre una copia con `JohnsonRules`. Se recalcula automáticamente cada vez que cambia la estructura del grafo.

Elemento	Descripción
`_service`, `_lastResult`	Servicio y resultado actuales.
`_objective`, `_from`, `_to`	Lo que eligió el usuario (máx./mín., origen, destino).
`_pathEdgeIds`	Aristas del camino a resaltar (+ la distancia como último elemento).
`_message`	Mensaje informativo o de error.
`initState()` / `dispose()`	Se suscribe/desuscribe con `_onGraphChanged` y ejecuta el cálculo inicial.
`_onGraphChanged()`	Solo recalcula si cambió la *firma* estructural (no por cambios de color o nombre).
`_structureSignature()`	Texto con ids de nodos y datos de aristas (id, origen, destino, peso, dirigida).
`_fail(message)`	Limpia resultado y selección y muestra el error.
`_runJohnson()`	Valida (hay aristas, reglas), busca un nodo inicio y fin, crea `JohnsonService`, ejecuta, comprueba que la diagonal sea 0 (si no, hay ciclo) y actualiza el estado. Llama a `_pathUpdate`.
`_pathUpdate()`	Pide `service.path(...)` con la selección actual y actualiza el resaltado o el mensaje «No path from X to Y».
`_buildTextPath()`	Arma el texto `A -> C -> F = 12.0` recorriendo el `Set` del camino al revés y usando el último elemento como total.

`build()`	Lienzo con camino resaltado (morado para máximo, amarillo para mínimo) y holguras, desplegables From/To, botón segmentado Maximize/Minimize, botón M[x] (abre `JohnsonMatrixScreen`), menú y Apply/Undo/Help.

10.5 Matrices

Pantalla	Descripción
`MatrixScreen({square})`	`matrix_screen.dart` Muestra la matriz de adyacencia como `Table` con scroll en ambas direcciones: encabezados de fila/columna, pesos, columna «Attributes leaving» (suma) y «Grade» (grado), y filas finales «Attributes arriving» y «Grade». Con `square: true` usa `padToSquare` y pinta los encabezados ficticios en gris. Métodos: `_fmt` (enteros sin decimales, si no 2 decimales), `_headerColor`, `_cell`.
`JohnsonMatrixScreen`	`johnson_matrix_screen.dart` Dos pestañas, MAX y MIN, con la matriz de distancias entre todos los pares. ∞ se muestra como «—». Métodos: `_fmt`, `_cell`, `_table(matrix)`.

10.6 `ChatScreen` (asistente IA)

`lib/screens/chat_screen.dart`

Elemento	Descripción
`_messages`, `_isLoading`	Historial local de la conversación y si se espera respuesta.
`_sendMessage()`	Agrega el mensaje del usuario, llama a `AiService.sendMessage` con el historial previo y el resumen del grafo del provider más cercano (el principal o la copia, según desde dónde se abrió), y agrega la respuesta o un mensaje «⚠» de error.
`_scrollToBottom()`	Tras el siguiente frame, anima el scroll al final.
`_MessageBubble`	Burbuja alineada a la derecha (usuario, azul) o izquierda (IA, celeste).
`_ThinkingBubble`	Burbuja «Thinking…» con indicador de carga.

11. Pantallas de ordenamiento

`selection_sort.dart` · `insertion_sort.dart` · `shell_sort.dart` · `merge_sort.dart`

Las cuatro pantallas tienen la **misma estructura**; solo cambia el algoritmo y los índices que se resaltan. Cada valor es un `Bolo` cuya altura (`escala`) representa el número.

Partes comunes

Elemento	Descripción
`maxBolos` (20), `escalaInicial` (0.3), `incremento` (0.08)	Límite de bolos y cómo se generan los tamaños: bolo i tiene escala `0.3 + i·0.08`.
`pausaComparar`, `pausaIntercambio` / `pausaEscribir`	Pausas de la animación (120 ms / 350 ms / 300 ms).
`_cantidad`, `_escalas`	Número de bolos (slider) y la lista de valores a ordenar.
`_ordenando`, `_fijos`	Si la animación está en curso (deshabilita controles) y cuántos bolos ya están en su lugar (verde).
`_cronometro`, `_reloj`	`Stopwatch` y `Timer.periodic` de 100 ms que redibuja el tiempo.
`_generarEscalas()`	Crea los bolos ordenados y reinicia el estado.
`_reiniciarEstado()`	Limpia índices resaltados y el cronómetro.
`_cambiarCantidad(v)`	Callback del slider.
`_mezclar()`	`shuffle()` de la lista.
`_resolver()`	Ejecuta el algoritmo de forma `async`: cada paso hace `setState` y `await Future.delayed(...)` para que se vea. Comprueba `mounted` tras cada pausa (por si el usuario sale de la pantalla).
`_colorDe(idx)`	Color de cada bolo según el estado del algoritmo.
`_numeroDe(escala)`	Convierte la escala en el número que se pinta (1 = el más pequeño).
`_tiempo` (getter)	Formato `mm:ss.d`.
`dispose()`	Cancela el `Timer`.

Diferencias por algoritmo

Pantalla	Cómo ordena	Colores
`SelectionSort`	Para cada posición `i` busca el mínimo en el resto (`j`) y lo intercambia con `i`.	Naranja = mínimo actual, amarillo = comparado, celeste = posición a fijar, verde = fijos.
`InsertionSort`	Toma cada bolo y lo desplaza a la izquierda mientras su vecino sea mayor.	Naranja = bolo en inserción, amarillo = vecino comparado, verde = parte ordenada.

`ShellSort`	Inserción con saltos gap = n/2, n/4, ..., 1.	Naranja / amarillo; verde solo al final.
`MergeSort`	`_mergeSort(izq, der)` divide recursivamente; `_mezclarTramos(izq, medio, der)` copia ambas mitades y las intercala en orden.	Naranja = posición escrita, amarillo = tramo que se mezcla; verde al final.

12. Tema y colores

`lib/theme/app_colors.dart`

`AppColors` reúne todas las constantes de color para no repetirlas. Grupos: aristas (`leftRightEdge` naranja, `rightLeftEdge` turquesa, `selfLoopEdge` morado, `previewEdge`), etiquetas de nodos (`lightNodeLabel`, `darkNodeLabel`), asignación (`assignmentEdge` verde, `ghostNode`, `ghostNodeBorder`, `ghostEdge`, `noEdgeAssignment` rojo), Johnson (`maxJohnson` morado, `minJohnson` amarillo), chat (burbujas y placeholder), matriz (`headerBk`, `summaryBk`, `matrixBorders`) y `gridColor`. `themeSeed` existe pero `main.dart` usa `Colors.indigo` directamente.

13. Flujos completos paso a paso

Crear una arista

1. `GraphWidget.onPanDown`: `findNodeAt` encuentra el nodo bajo el dedo → `sourceNode`.
2. `onPanUpdate`: actualiza `currentDragPosition`; `GraphPainter` dibuja la línea azul de vista previa.
3. `onPanEnd`: busca el nodo bajo el punto final y llama a `provider.checkEdgeEndpoints` → `rules.checkEdgeEndpoints`.
4. Si es válido, `showEdgeDialog` pide el peso (y dirección si se permite).
5. «Save» → `provider.addEdge` → `rules.checkAddEdge` → `_pushHistory` → `Graph.addEdge` → `notifyListeners`.
6. `GraphWidget` se reconstruye (`context.watch`) y el pintor dibuja la arista nueva.

Resolver una asignación

1. `HomeScreen` o `GraphScreen` abre `AssignmentScreen` con una copia y `AssignmentRules`.
2. El usuario pulsa Minimize → `_runAssignment(true)`.
3. `buildMatrixData` (filas = orígenes, columnas = destinos).
4. `AssignmentService.prepare` pone penalizaciones donde no hay arista; `HungarianAssignment.solve` rellena a cuadrada y ejecuta Kuhn-Munkres.
5. `buildOverlay` crea nodos/aristas fantasma y los pares; la pantalla resalta las aristas y muestra la hoja con el total.
6. Si el usuario edita el grafo, `_listenerClear` borra el resultado.

Calcular la ruta crítica (Johnson)

1. Se abre `JohnsonScreen` con una copia y `JohnsonRules` (sin ciclos, solo dirigidas).
2. `_runJohnson` detecta el nodo inicio (sin entradas) y fin (sin salidas).
3. `JohnsonService.prepare` → matriz de costos; `Johnson1.solve` → Floyd-Warshall para máximos y mínimos.
4. `run` calcula `backwards` y `holgura` de cada arista.
5. `_pathUpdate` → `service.path` reconstruye el camino; se resalta y se muestra `A -> … = total`.
6. Cambiar From/To/objetivo solo vuelve a llamar a `_pathUpdate`; editar el grafo vuelve a ejecutar todo.

Deshacer

1. Cada cambio llama a `_pushHistory()`, que apila un `clone()` (máximo 20).
2. «Undo» → `undo()` reemplaza `_graph` por la última copia y notifica.
3. Cada `GraphProvider` tiene su propio historial: deshacer en una copia de trabajo no afecta al principal.

14. Observaciones y puntos a revisar

Detalles encontrados al leer el código. No bloquean el uso normal, pero conviene tenerlos presentes.

> **Posible error al borrar un nodo con doble toque.** En `graph_canvas.dart`, `onDoubleTapDown` hace `Text(message!)` después de `removeNode`. Como `removeNode` devuelve `null` cuando todo sale bien, el `!` lanza una excepción. Debería ser igual que en el borrado de aristas: mostrar el SnackBar solo `if (message != null)`.

> `isLoop` **es método, no getter.** Se llama `edge.isLoop()`. Si la intención era un getter, cambiar a `bool get isLoop => …` y actualizar las llamadas. Además, varias reglas siguen usando `fromId == toId` / `edge.sourceNodeId == edge.targetNodeId`.

> `Graph.findEdgeAt` **con bucles.** Tras revisar el bucle no hace `return`, y continúa con el cálculo de segmento con distancia 0 (división entre 0 → `NaN`). No rompe nada porque la comparación con `NaN` es falsa, pero sería más claro devolver/continuar.

> **Guardado sin** `await`**.** En `GraphStorageService.save` y `delete`, `_writeAll(all)` no lleva `await`; el mensaje «Saved» puede aparecer antes de que termine la escritura.

> `JohnsonScreen._runJohnson`**.** Si `validateGraph()` devuelve errores llama a `_fail` pero no hace `return`, así que continúa ejecutando.

> `JohnsonRules.checkEdgeEndpoints`**.** La lista `edges` se acumula en cada iteración del BFS y se vuelven a encolar destinos ya vistos. Funciona (el `visitedNodes` lo frena) pero es ineficiente en grafos grandes; bastaría recorrer solo `graph.findEdges(nodeId)` en cada paso.

> **Código sin usar.** `johnson.dart` / `johnson_result.dart`, `GraphHelp`, el enum `Mode`, `AppColors.themeSeed` y `AppViewport` (zoom/pan) no se usan hoy. Decidir si se integran o se eliminan.

> **Nombres.** `Johnson1` en realidad usa Floyd-Warshall; `bestEdgeIdsMIn` tiene una mayúscula de más; el parámetro `tap` de `edgeExists` es el nodo destino.

15. Cómo agregar cosas nuevas

Quiero…	Pasos
Agregar un campo a `Node` o `Edge`	1) Añadirlo a la clase. 2) Copiarlo en `Graph.clone()`. 3) Agregarlo en `toJson()` y en `fromJson()` con un valor por defecto (para grafos guardados antiguos).

Un nuevo modo con reglas propias	1) Crear `MisReglas extends GraphRules` y sobrescribir solo los `check…` necesarios (llamando a `super` primero). 2) Crear la pantalla con un `mainProvider`. 3) Abrirla con `_pushWithCopy(context, MisReglas(), (main) => MiPantalla(mainProvider: main))`.
Una tarjeta en el menú principal	Añadir un `_Method` a `_methods` en `home.dart`.
Un concepto en el acordeón	Añadir un `_Concept` a `_concepts`.
Un nuevo algoritmo de ordenamiento	Copiar una pantalla de ordenamiento existente y reescribir `_resolver()` y `_colorDe()`.
Un color nuevo	Agregarlo como constante en `AppColors` en lugar de escribirlo en la pantalla.
Un paso nuevo en el tour	Dar una `GlobalKey` al widget y añadir un `TourStep` en el `_startTour()` de la pantalla.
Que la IA sepa algo nuevo	Editar `assets/docs/app_knowledge.md`.